

Worksheet 3: MPI

The final and also my favourite worksheet. This time, no benchmarks or analysis, it is all about the code! That said, even if a task doesn't explicitly ask for it, you should still keep performance in mind. Don't just aim for code that works—aim for code that works well. Apply all the performance insights you have picked up from the previous worksheets!

In this worksheet, you will once again start with a serial implementation of our heat equation solver. Your mission: transform it into a fully parallel version that can run efficiently on modern supercomputers, which (as you know) typically rely on distributed memory architectures.

Along the way, the worksheet will guide you through implementing a parallel distributed vector—a vector whose data is spread out across multiple memory spaces on different processes. In the context of the heat equation, this means introducing the concept of ghost elements, and implementing their exchange using MPI.

I won't sugarcoat it—this is probably the most challenging worksheet from an implementation standpoint. But it is also the most rewarding. Seriously, having a working MPI implementation by the end is nothing short of awesome.

I know things can get tricky, so please don't hesitate to reach out if you get stuck or something isn't clear. I am always happy to help!

Now let's dive in. I hope you enjoy working on this worksheet as much as I enjoyed putting it together for you!

Getting Started

Once opening the project folder in VSCode you should be prompted to reopen it in the container. If not, you can do this manually by clicking on the green icon in the bottom left corner of VSCode and selecting "Reopen in Container". After a short moment the container environment should be ready. Please note, that in case you have not yet setup the container image or anything changed in the docker file this might take a bit longer.

You find detailed installation instructions in the README provided in the project's root folder. If you run into any trouble, just ask — we are happy to help!

Code Structure

The project consists of five main folders:

- **applications/** – Contains the executable for the heat equation solver. Feel free to add applications if you want to experiment with your own code.
- **include/i2php/** – Header files for your classes and logic.
- **source/** – The corresponding implementation (.cpp) files.

- **tests/** – Contains unit tests which you can run to verify the correctness of your implementations. See the README for more details. Also feel free to add your own tests. Writing tests is a great way to deepen your understanding of the code. You might wanna have a look at test-driven development and GoogleTest for this. Note, that for this worksheet, the tests are almost exclusively designed to only test the final result. Although it is not explicitly part of the assignment, we highly recommend to write your own tests for the individual components you implement. This will help you to catch bugs early and make debugging easier.

Submission

Please submit your solution until Wednesday, July 26th, 2026 11:59 PM by opening a merge request of your developing branch into the main branch of the repository. The merge request should include the following:

- The modified code files including all required changes for the worksheet.
- A short PDF document summarizing the implementation strategy, i.e., what the different components are doing. Basically, a short summary of the explanations in this worksheet. In addition, you can include any problems you encountered and how you solved them. This is not only helpful for us to understand your solution, but also for you to reflect on your own work.

Today's Worksheet Tasks

The worksheet consists of four parts that will guide you through all the necessary modifications to build a fully functional, MPI-parallelized heat equation solver. In the provided code skeleton, you will find a basic MPI implementation that currently *only* works when running a single MPI process. Your task: extend it to support an arbitrary number of processes.

It is important that you tackle the tasks in order, as they build on one another. Each step introduces concepts and code needed for the next—so no skipping ahead!

Since each task involves some theoretical background, we won't dive into all the details here. Instead, those will be explained in the specific task descriptions that follow. For now, here are a few general notes to keep in mind. In Task 1, you will implement linear algebra vector functionality similar to what you have seen in previous worksheets but this time using MPI. It is a great warm-up to get your hands dirty with distributed data structures.

Although you can dive right into Task 1, I highly recommend reading through the entire worksheet first and familiarizing yourself with the codebase before starting your implementation. As always, the code framework includes tests to help you verify your solutions. However, the tests for Tasks 2, 3, and 4 check the full heat equation solver, so you will need to finish all the parts before those tests give meaningful results. As mentioned earlier, we highly recommend to implement your own tests for the individual components as this makes debugging significantly easier.

One more tip: it might be a good idea to get everything working in one spatial dimension before attempting the higher-dimensional cases (2D and 3D). It will make your life easier and debugging more manageable.

And finally, a word about bonus points: as you might have noticed while flipping through the worksheet, there are no explicit “bonus tasks.” But don’t worry—you still have a chance to earn bonus points! The worksheet includes a total of 44 points, but grading will be based on only 40 points. So there is some extra credit hidden in plain sight.

Task 1: Parallel Distributed Linear Algebra Vector (8 Points)

We kick off this worksheet by upgrading our familiar linear algebra vector class into a parallel, distributed version that works nicely with MPI. As you may recall, MPI is designed for distributed memory architectures, meaning each process has its own private memory space. No sneaky peeking into your neighbor’s memory allowed! So, when you are working with large vectors—say, with millions or even billions of elements—you will want to not only distribute the computations, but also the data itself. Each process should be responsible only for the part of the vector it actually operates on. This reduces both computation time (thanks to parallelism) and memory usage (since each process stores only a portion of the data). Neat, right?

Getting Started: Where to Look

Let’s first have a closer look at the provided code structure. To begin, take a look at the file `include/i2php/linear_algebra/parallel_distributed_vector.hpp`, which contains the interface of our parallel distributed vector class. The actual implementation belongs in `source/linear_algebra/parallel_distributed_vector.cpp`. As usual, the header file provides detailed documentation for each function.

It might be a good idea to read through the header first to get a feel for the interface. Don’t worry if you don’t yet understand every function—many of them will become clearer as we progress through this worksheet.

Your Implementation Task

In this task, we will focus exclusively on the core linear algebra operations for our distributed vector—similar to what you implemented in previous worksheets, but now adapted to the parallel world. The functions you will need to implement are

- `double dot_product(const ParallelDistributedVector &other) const;`
- `ParallelDistributedVector operator+(const ParallelDistributedVector &vec) const;`
- `void operator+=(const ParallelDistributedVector &vec);`
- `ParallelDistributedVector operator-(const ParallelDistributedVector &vec) const;`
- `void operator-=(const ParallelDistributedVector &vec);`

- `ParallelDistributedVector operator*(number num) const;`
- `void operator*=(number num) const;`
- `double l2_norm() const;`

One thing to note: unlike in previous worksheets, the `ParallelDistributedVector` class is not templated over the data type. We simply work with `double`. In larger projects, templating for flexibility is essential, but since we are not benchmarking or generalizing code today (yay, just coding!), we have kept things straightforward for the sake of clarity.

Working with Compatible Vectors

When working with two vectors—for example, in a dot product or addition—you may assume that they are *compatible*. In our context, that means both vectors store the same range of global indices on the same processes. (Strictly speaking, this includes ghost values too, but we will dive into that in more detail later.)

It is worth pointing out that this compatibility assumption is specific to our codebase. Other libraries, like the finite element library `deal.II`, define compatibility more loosely—for example, requiring only that the number of locally owned entries match, not necessarily the specific indices.

Still, it is always good practice to check compatibility explicitly, to catch bugs early. For that, you can use the already-implemented method `assert_compatibility(...)`, which verifies that two vectors are laid out the same and aborts the program if not. It is a small check that can save you a lot of debugging time.

Global vs. Local Indices

Before you dive into implementation, it is crucial to understand how data is distributed across processes. There are two kinds of indices you will deal with:

- **Global indices:** refer to the full vector layout (as you would write it down mathematically) starting at 0 and going up to the last element.
- **Local indices:** refer to the part of the vector stored *on the current process*.

Accessing a global index might require inter-process communication, which is relatively expensive. So unless you really need it, stick to local indices for fast, direct access. The local elements are stored in a private member:

- `std::vector<double> local_elements;`

You can access them using either `operator[]` or the member function `local_element()`, whichever you find more convenient.

Testing Your Implementation

Once you have written the required functions, it is time to test your code. We have provided a test file at:

- `tests/parallel_distributed_vector_test.cpp`

You can run the tests using

```
mpiexec -n 8 --oversubscribe ./tests/parallel_distributed_vector_test
```

Note, that the `-oversubscribe` flag is optional, depending on the number of physical cores your machine has—check the course script for details.

A quick heads-up: the tests are only designed to run with up to 15 MPI processes. Using more will cause a segmentation fault, due to the way the tests are constructed. So, for now, keep it under 16 processes.

Wrap-up

You now have all the theoretical background and tools needed to implement your first building block of the MPI-based solver. This parallel distributed vector will serve as the mathematical backbone for everything that follows—so take your time, test thoroughly, and enjoy the challenge.

Task 2: MPI Partitioner (18 Points)

Having implemented the linear algebra functionality of our parallel distributed vector, you should now be well warmed up. In this task, we focus on the true heart of the vector class: the *MPI partitioner*. This component is responsible for organizing and executing all communication between processes. It keeps track of who needs to send what to whom, what to receive from where, and even performs the communication itself. Think of it as the beating heart of the distributed data structure.

The interface for the partitioner class is located in `include/i2phpp/util/partitioner.hpp`, and its implementation in `source/util/partitioner.cpp`.

Why Ghost Values?

Before diving into the implementation, we need to take a step back and understand why we need such a specialized partitioner in the first place.

In distributed memory systems, each process owns a portion of the global data. In our case, this means a set of *locally owned degrees of freedom*. However, when performing finite difference computations—such as those used in our heat equation solver—we often need access to neighboring values. If a degree of freedom lies on the boundary of a local subdomain, then some of its neighbors may not be locally owned. To solve this, we introduce the concept of *ghost values*: data elements that are not locally owned but are still required for local computations. These values are received from the processes that

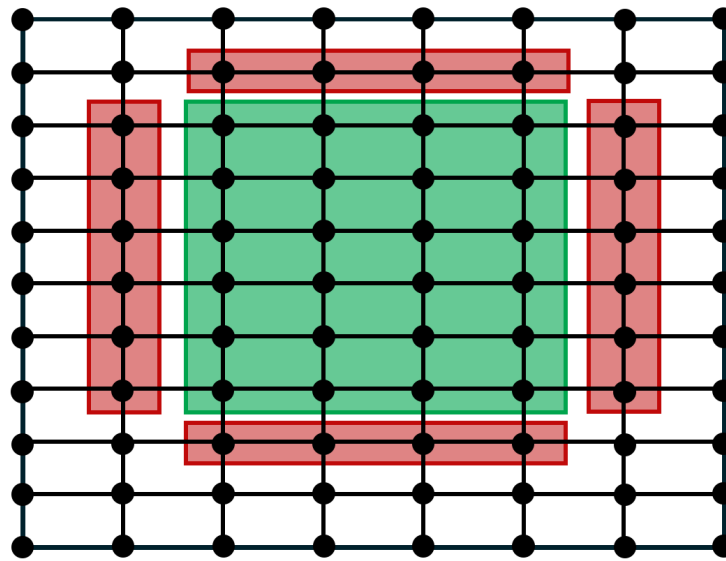


Figure 1: Illustration of locally owned and ghost values. Degrees of freedom in green are locally owned, while those in red are ghost degrees of freedom. Ghost values are required to compute finite differences at the boundaries of locally owned domains.

actually own them and stored as read-only copies. As a result, each process maintains two sets of data:

- **Locally owned elements**, which it computes and updates.
- **Ghost elements**, which it receives and uses in calculations but never modifies.

Together, these form the set of *locally relevant* elements.

Efficient Ghost Exchange

Once ghost values exist, they must also be updated regularly. Any time a process changes a value that another process intends to use as a ghost element, it must notify that process. This communication must be correct—but also efficient. We want to minimize unnecessary communication to reduce overhead. Managing this is the key role of the MPI partitioner.

To manage the sets of indices cleanly, we use the `IndexSet` utility class. You can find it in `include/i2php/util/index_set.hpp`; the implementation is in the corresponding source file. You do not need to modify this class, but you should use its functionality to make your implementation cleaner and more maintainable.

The `Partitioner` class internally stores both the locally owned and ghost indices as `IndexSet` objects. You may also extend this class with helper methods if needed. For simplicity, all locally owned indices are assumed to form a contiguous range.

Determining the Communication Pattern

The first step in implementing the MPI partitioner is to set up the communication pattern between processes. This is done in the function

```
void determine_communication_pattern();
```

This function determines which data elements need to be exchanged, and between which processes. More precisely, it fills the following two vectors

- `std::vector<std::pair<int, IndexSet>> export_targets_and_indices;`
- `std::vector<std::pair<int, IndexSet>> import_targets_and_indices;`

Each element of the vectors is a pair consisting of

- An integer MPI rank, and
- An `IndexSet` representing the indices to be sent or received.

In `export_targets_and_indices`, the rank refers to a process to which the current process must send data. In `import_targets_and_indices`, the rank identifies a process from which the current process will receive ghost values. You can choose to use either global or local indices—whichever you find more intuitive. Both approaches are valid in this context.

To guide you through this task, the function is already annotated with minor comments. The overall idea consists of three major steps:

1. **Ghost request phase:** Each process broadcasts a message listing the global indices of the ghost values it needs from other processes. This acts as a request, informing others of which data is required.
2. **Ownership check phase (local):** After receiving ghost requests, each process compares the requested indices with its own locally owned elements. It then prepares the vector `export_targets_and_indices`, indicating which data to send to whom.
3. **Final scatter:** Using the information gathered, each process constructs the vector `import_targets_and_indices` to determine from which ranks it will receive ghost values. This step finalizes the communication pattern.

Once these steps are complete, the partitioner knows exactly what data to send and receive and is ready for the actual communication phase.

Exchanging Ghost Values

Now that the communication pattern has been established, the next task is to implement the actual data exchange. In the `ParallelDistributedVector` class, you will find the function

```
void update_ghost_values();
```

This function is responsible for updating ghost values across all processes. Internally, it delegates the work to two methods of the `Partitioner` class:

- `export_to_ghosted_array_start(...);`

- `export_to_ghosted_array_finish(...);`

These methods are intentionally split to support non-blocking communication. This allows you (in theory) to perform computations while data is being transferred—although the current implementation doesn’t take advantage of this.

Now that we have set up who should talk to whom, let’s take a closer look at how the actual data transfer begins. The first function you will work with is:

```
template <std::ranges::contiguous_range ContigView>
void Partitioner::export_to_ghosted_array_start(
    const ContigView &locally_owned_array,
    ContigView &ghost_array);
```

This templated function initiates non-blocking MPI communication. The template type `ContigView` must behave like a standard contiguous container—meaning it supports `begin()`, `end()`, and `data()`. In essence, this function does two things:

- It sends out the required entries from the locally owned vector to the appropriate processes.
- It posts non-blocking receive requests for the ghost values expected from other processes.

To manage these asynchronous operations, use the `Partitioner` class’s `request` member variable, which stores a collection of `MPI_Request` objects. This allows your code to continue executing while data is still being transferred in the background.

Once all send and receive operations have been posted, the next step is to wait for them to complete and finalize the data transfer. This is handled by

```
template <std::ranges::contiguous_range ContigView>
void Partitioner::export_to_ghosted_array_finish(ContigView &ghost_array);
```

This function blocks until all ongoing communication operations are done. Once completed, it places the received ghost values into their corresponding positions within the `ghost_array`, making them ready for use in further computations.

Wrapping Up

Once you have implemented both the communication pattern and the ghost value exchange, your MPI partitioner will be fully operational. This component is critical for enabling efficient parallel computations in your solver—so it is worth taking the time to get it right.

If things seem complex at first, don’t worry. Take it step by step, refer to the comments in the code, and always test your implementation thoroughly.

You’re building the communication backbone of a distributed solver—a big step forward in your MPI journey!

Task 3: DoF-Handler (8 Points)

Excellent work so far! You have already completed the most complex part of today's worksheet—the MPI partitioner. With the foundation laid, it is time to gradually shift our focus toward the heat equation solver.

As with most things in life, at some point, someone (your boss, a funding agency, or a skeptical colleague) will ask why all this technical effort is worthwhile. Having a robust MPI framework is great—but if it is not applied to solve a real-world problem, it can be difficult to justify the time and resources spent. So, let's make sure we are not just having fun implementing (though we are!)—we are also solving something meaningful.

We will now put the `ParallelDistributedVector` and its MPI partitioner to use in the heat equation solver.

Transitioning to the Parallel World

The first hurdle in porting the serial heat equation solver to an MPI-parallel version is deciding how to distribute the computational work. More precisely, we need to determine which process owns which degrees of freedom (DoFs), and based on that, figure out which ghost elements each process needs access to.

There are various strategies to distribute DoFs across MPI ranks, but they all aim to achieve two primary objectives:

1. **Minimize communication overhead:** Fewer ghost elements and fewer communication partners lead to faster and more scalable execution.
2. **Balance the computational load:** Each process should own roughly the same number of elements. In more complex equations, where some DoFs require more computation than others, achieving perfect balance can be extremely challenging.

For our worksheet, we use a simplified—but clear and effective—strategy. While it may not be the most sophisticated approach, it allows us to focus on understanding the concepts without getting lost in technical complexity.

1D Strategy: A Clean Split

In the 1D case, the splitting is straightforward: divide the domain into n equally sized continuous chunks of DoFs, assigning one chunk to each MPI process. The difference in locally owned elements between processes is at most one, ensuring near-perfect load balancing. Each process ends up with exactly two ghost elements and communicates with exactly two neighboring ranks. For this specific use case—the 1D heat equation—this is likely the optimal DoF distribution.

Higher Dimensions: Slices and Simplicity

In 2D and 3D, things naturally get more complicated. Ideally, we would want to split the domain into squares (or cubes) to minimize ghost elements and maintain balance. But implementing such strategies for arbitrary numbers of DoFs and MPI ranks is far from trivial.

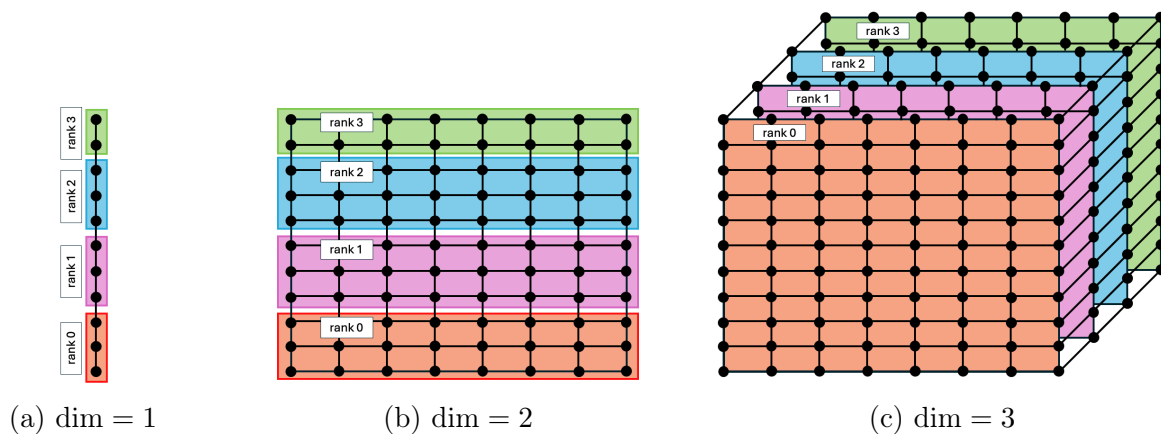


Figure 2: Illustration of the DoF distribution strategy across MPI processes for 1D, 2D, and 3D domains.

So instead, we extend our 1D logic in a simple, consistent way. We always split the domain along the x_{dim} axis—the last dimension. For example:

- In 2D, each process gets a strip of rows (in the y direction) spanning the full x -axis.
- In 3D, each process gets a slab in the z -direction, covering the full xy -plane.

This approach isn't optimal, but it is predictable and easy to implement. The visualizations in Figure 2 should help you understand how the slices are assigned across dimensions.

Implementation Details

The splitting strategy described above is already implemented in the following function:

```
void i2phpp::DoFHandler<dim>::initialize_dof_vector(
    i2phpp::ParallelDistributedVector &dof_vector,
    const double init_value) const;
```

This function receives a distributed vector and initializes its partitioning using our 1D-based slicing method. However, there is one flaw in this current setup.

The Problem: Too Many Processes, Not Enough Work

Picture this: an ambitious team of engineers is using your code to simulate heat transfer in a cutting-edge rocket engine. They are experts in fluid dynamics and thermodynamics, but when it comes to MPI... let's just say they would prefer not to know what those three letters stand for. Eager to get faster results, they launch your simulation on a supercomputer—using every core they can get their hands on.

And then it crashes.

Why? Because there are more MPI processes than there are chunks of the domain to assign. And while that might be totally obvious to you (a seasoned parallel programmer at this point), it is not exactly intuitive for someone who is thinking about rocket nozzles,

not ghost elements.

We don't want our users to end their supercomputing session confused and frustrated, so let's make our code a little smarter and more forgiving. Here is how we will handle this situation:

- First, check whether the number of MPI ranks exceeds the number of slices available in the splitting direction.
- If yes, create a new MPI communicator that includes only the first n ranks—where n is the number of available slices.
- All remaining ranks should be assigned `MPI_COMM_NULL` and happily sit idle (maybe grab a coffee).

This adjustment ensures that your simulation runs robustly, even if someone gets carried away with the number of MPI processes. It won't make things faster—but we will let our rocket scientist believe it does.

You can implement this logic using the `ParallelDistributedVector::reinit(...)` function:

- For active ranks: use `reinit(...)` with the correct partitioning and the newly created communicator.
- For idle ranks: initialize an empty `ParallelDistributedVector` using the communicator `MPI_COMM_NULL`.

Important: You only need to modify the `initialize_dof_vector` function—everything else in the codebase is already set up to support this behavior.

Hint: Still on the hunt for a bonus project idea? Try implementing smarter DoF splitting strategies! MPI's cartesian topologies are a great place to start—and way more fun than they sound.

Task 4: Heat Operator (10 Points)

With everything else in place, it is time to bring it all together and finally put our parallelization framework to work in the `HeatOperator` class. Up to now, the class already includes basic MPI support—but there is a catch: it only gives correct results when using a single process. Technically, it still “works” with multiple processes, but each one independently solves the entire domain. That is not quite the parallelism we are aiming for—it is more like every worker baking the same cake from scratch instead of dividing the recipe.

Note, that in this worksheet we will only work on the matrix-free implementation. The matrix-based framework has already been removed from the code base and will not be considered further.

Initializing the DoF Vector

As a first step, you will need to properly initialize the DoF vector—specifically, the two vectors `current_solution` and `old_solution`. Thankfully, you don't need to reinvent the wheel here. Simply use the `reinit_dof_vector()` function from the previous task to set them up according to the correct partitioning.

Heat Equation with Explicit Time Stepping

To compute the solution in parallel using the explicit Euler time stepping scheme, you will need to modify the following member functions to work with our `ParallelDistributedVector`:

- `void i2phpp::HeatOperator::advance_time_step(double current_time, double current_time_increment);`
- `void i2phpp::HeatOperator::apply_boundary_conditions();`
- `double i2phpp::HeatOperator::matrix_free_stencil(unsigned global_index, const ParallelDistributedVector& solution) const;`

The good news: everything else in the codebase is already MPI-ready. You just need to connect the dots here. To verify your implementation, make use of the tests provided in `tests/heat_equation_tests`. A correct solution should pass the tests regardless of the number of MPI processes—whether you are using 2, 20, or 100.

A few testing tips:

- Don't rely on one fixed number of processes—test with different values! This helps catch subtle bugs.
- Run at least one test with more than 25 processes to also validate the communicator logic you implemented in Task 3.

Heat Equation with Implicit Time Stepping

Now that the explicit version of the solver is running in parallel, it is time for the final challenge: adapting the implicit matrix-free solver to MPI as well. Luckily, this is just a variation of what you have already done, and the changes are quite similar.

To finish the job, modify the following two functions:

- `void i2phpp::HeatOperator::advance_time_step(double current_time, double current_time_increment);`
- `double i2phpp::HeatOperator::jacobi_update(unsigned global_index, double time, double current_time_increment, const ParallelDistributedVector& old_iter_vector, const ParallelDistributedVector& rhs) const;`

As before, use the same test file to confirm correctness. One helpful hint: take special care with ghost value synchronization. If in doubt, try updating ghost values in multiple places—then remove the redundant calls after confirming functionality.

Final Comments

And that's it—you have completed the final task of the semester! Huge congratulations are in order. You have officially crossed into the world of parallel computing.

Let's take a quick look back: you began with a simple serial heat equation solver, then used SIMD to unleash data-level parallelism on a single core. With C++ threads, you explored shared memory, and OpenMP made that journey even smoother. Finally, you scaled up to distributed memory systems with MPI and built your own parallel distributed vector class—an essential building block in any high-performance numerical library.

It has been a wild ride. I hope you enjoyed this journey into the world of performance engineering as much as I enjoyed guiding you through it.

To close things off, here is one final analogy:

*SIMD is your lightsaber,
C++ threads are your loyal allies,
OpenMP is your magic spell,
and MPI... your intergalactic postman.*

May the cores always be with you!